Applying the Unscented Kalman Filter (UKF) to Predict Stock Prices

Simon Leung 12 min read · Nov 8, 2024

Besides self-driving cars, the Unscented Kalman Filter can also be used for... *

New Update (2026–01–23): revised Python code.

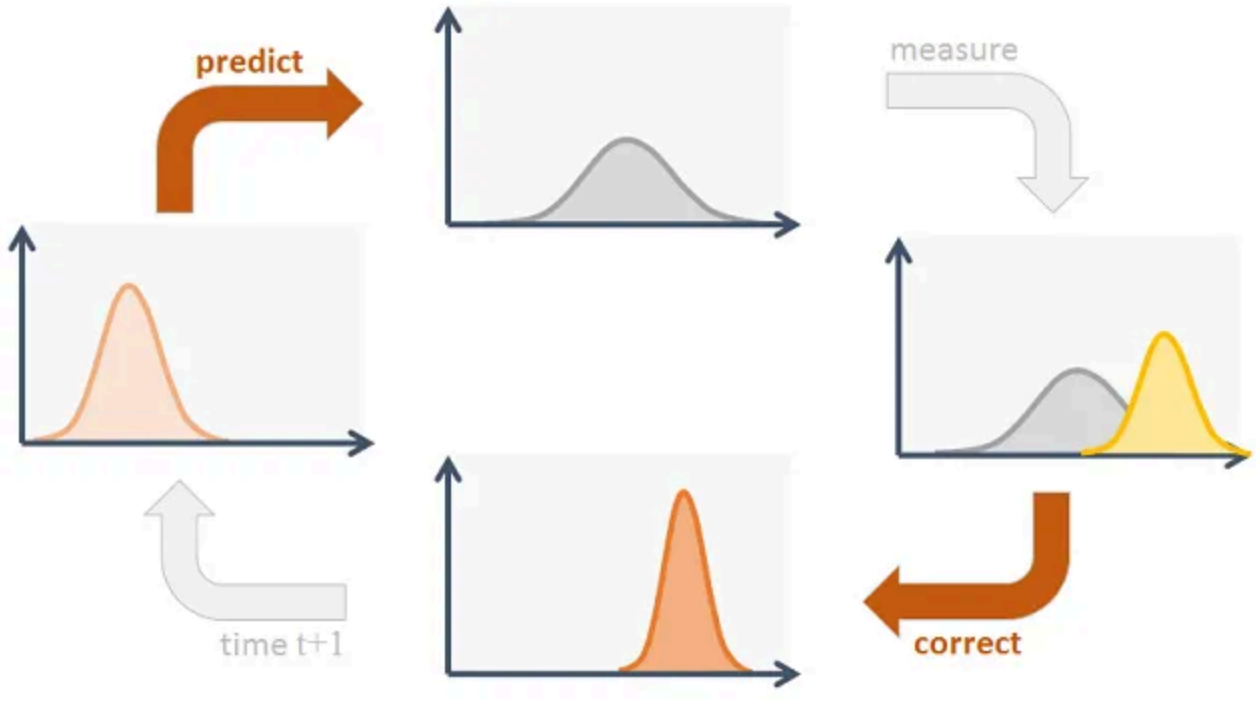


Image from [1]

Imagine you are navigating and tracking systems to estimate the position and velocity of moving objects, where observed measurements such as those from sensors (GPS, radar, etc.) are noisy and not perfectly accurate. The Kalman Filter is an optimal estimator for linear systems, predicting system states in the presence of uncertainty (Gaussian noise).

Based on the current position and speed of a car and the mathematical model of the car's motion, the filter predicts the car's state at the next 'future' time-step, which also contains some uncertainty. When a new measurement of the car's actual position and velocity is received, the Kalman Filter updates the predicted state using the latest information. This is done by combining the predicted state and the new measurement in a way that minimizes uncertainty. However, when dealing with nonlinearity, the standard Kalman Filter struggles because it relies on linear approximations.

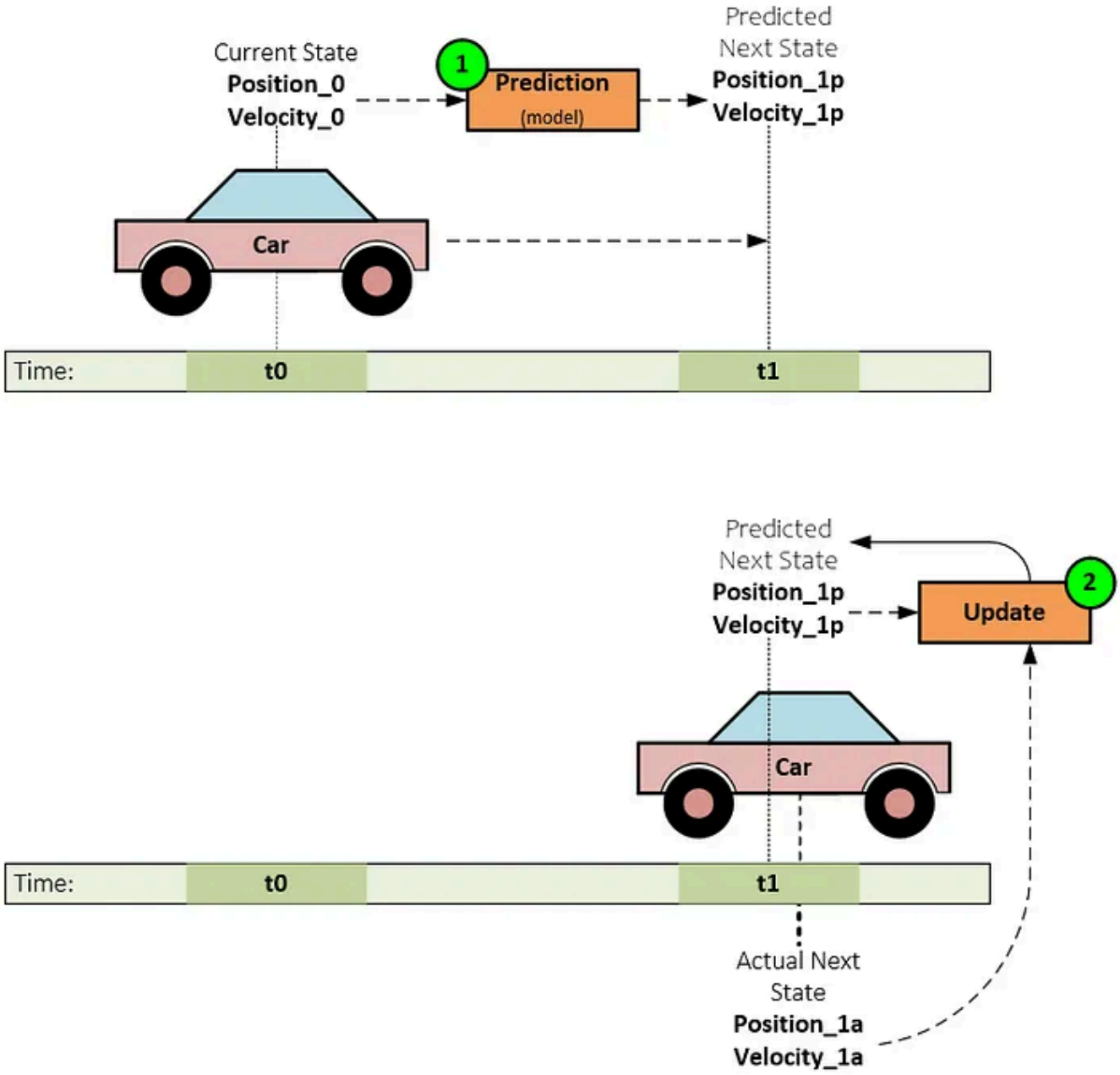


Image from [2]

To address nonlinearity, extensions such as the Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF) are used. The UKF uses the Unscented Transform (UT), which approximates the nonlinear transformation by applying it to a carefully selected set of sample points (called sigma points). This approach captures the mean and covariance of the transformed variables more accurately than linearization methods like the Extended Kalman Filter (EKF).

Additional articles on *Unscented Kalman Filter*:
The Unscented Kalman Filter: Anything EKF can do I can do it better!
Unscented Kalman Filter
The Unscented Kalman Filter, simply the best! Python code
What is a Kalman filter and why is there an unscented version?
Kalman Filter, Extended Kalman Filter, Unscented Kalman Filter

How the Unscented Kalman Filter (UKF) works:

1. Initialization: You start with an initial estimate of the system's state and its uncertainty (covariance matrix).
2. Prediction: You use a mathematical model of the system to predict the next state, based on the current estimate and the system's dynamics.

3. Unscented Transformation: You create a set of “sigma points” around the predicted state. These sigma points are carefully chosen to capture the uncertainty of the state. Think of them as a cloud of points around the predicted state.

4. Measurement Update: You take a measurement of the system (e.g., GPS reading). You then use the measurement model to compute the expected measurement for each sigma point.

5. Weighted Average: You compute a weighted average of the expected measurements, using the sigma points and their corresponding weights. This gives you an updated estimate of the system’s state.

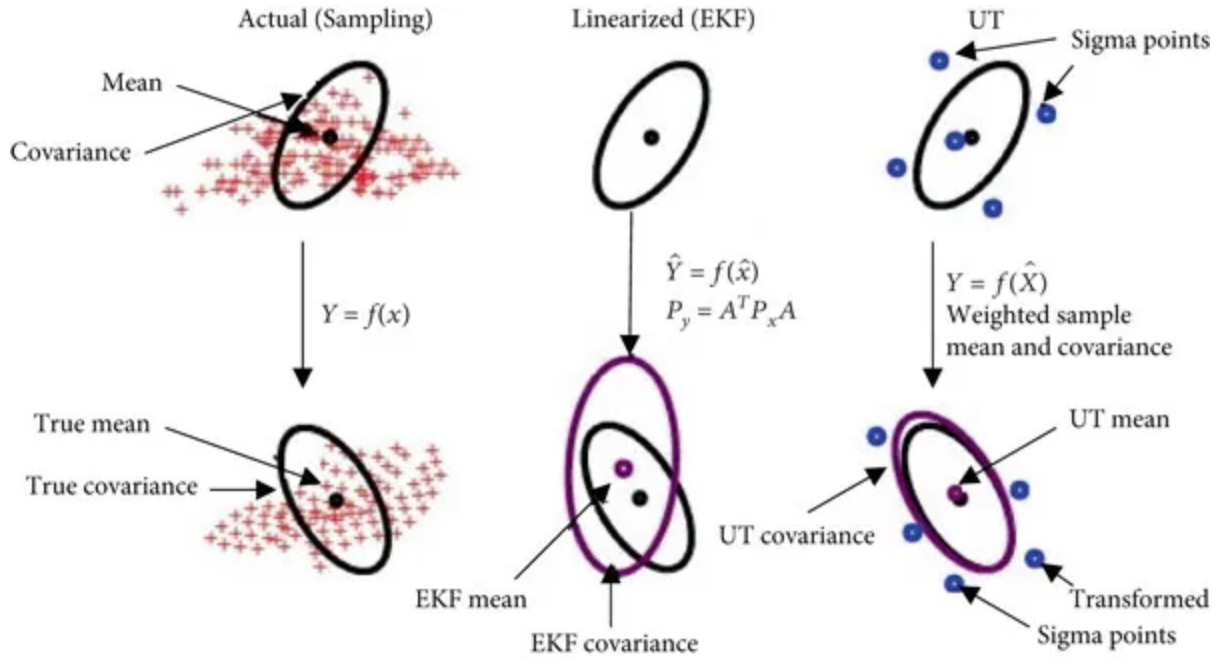


Image from [3]

6. Covariance Update: You update the covariance matrix of the state estimate, using the weighted average and the measurement noise covariance.

7. Repeat: You repeat steps 2–6 for each new measurement, updating the state estimate and covariance matrix.

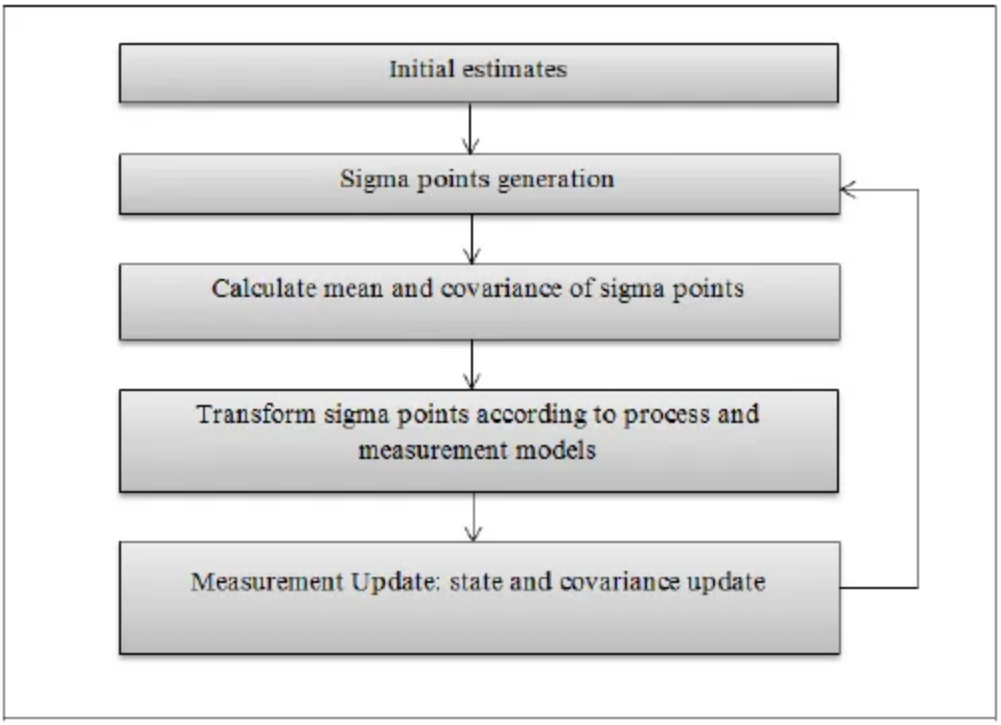


Image from [4]

Now let's consider:

- **Current Stock Price:** This is like the **position** of the car. It tells you where the car (or stock) is at a specific moment in time.
- **Stock Volatility:** This is like the **velocity** of the car. It measures how quickly and unpredictably the car's (or stock's) position is changing over time.

So, just as knowing the position and velocity of a car gives you a good understanding of its movement, knowing the current stock price and its volatility helps you understand both the value and the risk associated with the stock.

[Full revised source code here] Thanks for a reader — Gregory Aharonian

Install required library:

```
# https://filterpy.readthedocs.io/en/latest/index.html
pip install filterpy
```

Define state transition function and measurement function:

The state transition function, f(x,t), (also known as the process model) represents how the internal state of the system changes over time. In simpler terms, it describes the dynamics of the system in the absence of measurement updates.

The measurement function, h(x), (also known as the observation model) defines the relationship between the hidden state and the observed measurements. It maps the internal state of the system to the observed measurements we can actually obtain.

```
# Define state transition function
def fx(x, dt):
    # Simple model assuming constant velocity, x[0] is price, x[1] is price chan
    return np.array([x[0] + x[1] * dt, x[1]])

# Define measurement function
```

```
    def hx(x):
        return np.array([x[0]])  # We only measure the price for simplicity
```

The *MerweScaledSigmaPoints* function in the Python library FilterPy is used to generate sigma points for the Unscented Kalman Filter (UKF). This function helps in the Unscented Transform, a way of estimating the mean and covariance of a Gaussian distribution that undergoes a nonlinear transformation.

The parameters alpha, beta, and kappa are tuning parameters that control the spread and weighting of the sigma points.

- **Alpha (α)**: This parameter is typically set to a small positive value. Common values range from 0.001 to 0.1. It controls the spread of the sigma points around the mean. Larger values of alpha increase the spread of sigma points, which can capture more nonlinearity, but they also introduce more uncertainty.
- **Beta (β)**: This parameter is often set to 2, which is optimal if the state distribution is Gaussian. However, values can range from 0 to 4, depending on how much prior knowledge you have about the distribution. This parameter is not as critical to tune unless you know the distribution is not Gaussian.
- **Kappa (κ)**: A common choice for kappa is 0 or 3-n (where n is the state dimension). Kappa controls the secondary spread of the sigma points and is often used to fine-tune the filter. kappa has a more subtle effect than alpha and beta. Setting it to a low or negative value reduces the spread, making the estimate more conservative.

Implementation of the Unscented Kalman Filter (UKF), P, R, and Q are key matrices that represent different types of uncertainty and noise in the system.

1. **P (initial state covariance)**: This represents the initial uncertainty in the state estimation. A good starting point might be a small positive value like 0.01 to 10. If your initial state is known with high confidence, use smaller values (e.g., 0.01). For unknown states, use larger values.

2. Q (process noise covariance): This parameter models the process noise in the system, such as unexpected changes in the state.. Values typically range from 0.001 to 1. Larger values indicate greater uncertainty in the system dynamics. If your system experiences sudden changes or noise (e.g., due to market volatility), increase this value.

3. R (measurement noise covariance): This parameter models the measurement noise. Common values range from 0.01 to 1. Higher values indicate greater uncertainty in the measurements. If the data has more noise, increase R so the filter gives more weight to the model than the measurements.

Gather historical data:

```
# I use =GOOGLEFINANCE("AAPL","all",DATE(2020,1,1),DATE(2024,10,21), "DAILY")  o
# and download data save as AAPL.csv
stock_data = pd.read_csv('./AAPL.csv')
stock_data['Date'] = pd.to_datetime(stock_data['Date'])
stock_data.set_index("Date", inplace=True)
```

```
stock_data.info()

# Save last 15 points for predictions comparison
close_prices = stock_data[['Close']][:-15]
```

Perform a grid search to find the optimal parameter values based on the R^2-score:

```
# Split data into training and testing sets (80% training, 20% test)
train_data, test_data = train_test_split(close_prices['Close'].values, test_size
```

Note: Need to change the var number if necessary. My number is based on daily return of AAPL close price.

```
ukf.Q = Q_discrete_white_noise(dim=n_dim_state, dt=dt, var=0.0004) * Q
```

Split data into training and testing sets for Grid searching purpose.

```
from filterpy.kalman import UnscentedKalmanFilter, MerweScaledSigmaPoints
from filterpy.common import Q_discrete_white_noise
from sklearn.metrics import r2_score
from itertools import product



# UKF setup
dt = 1.0   # Assuming daily interval (or adjust as per your data frequency)
n_dim_state = 2  # price and velocity
```

```
n_dim_meas = 1  # only measuring price

# Grid search for best parameters
alpha_values = [0.001, 0.01, 0.05, 0.1]
beta_values = [1.0, 2.0, 3.0, 4.0]
kappa_values = [0, 1, 1]    # [0, 1, 3-n] (where n is the state dimension)

P_values = [0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
Q_values = [0.001, 0.01, 0.05, 0.1, 0.2, 0.5, 1.0]
R_values = [0.01, 0.1, 0.5, 1.0]


# Define state transition function
def fx(x, dt):
    # Simple model assuming constant velocity, x[0] is price, x[1] is price chan
    return np.array([x[0] + x[1] * dt, x[1]])

# Define measurement function
def hx(x):
    return np.array([x[0]])  # We only measure the price for simplicity

# Grid search
best_r2 = -np.inf
best_params = None

# Grid search over all parameter combinations
for alpha, beta, kappa, P, Q, R in product(alpha_values, beta_values, kappa_valu
    # Initialize sigma points and UKF with current parameter set
    sigma_points = MerweScaledSigmaPoints(n=n_dim_state, alpha=alpha, beta=beta,
    ukf = UnscentedKalmanFilter(dim_x=n_dim_state, dim_z=n_dim_meas, fx=fx, hx=h

    # Set covariance matrices for current parameter set
    # an identity matrix scaled by a constant
    ukf.P = np.eye(n_dim_state) * P
    #ukf.Q = np.eye(n_dim_state) * Q
    ukf.Q = Q_discrete_white_noise(dim=n_dim_state, dt=dt, var=0.0004) * Q   # v
    ukf.R = np.eye(n_dim_meas) * R


    # Initialize state (assuming initial price is known and velocity is zero)
    ukf.x = np.array([train_data[0], 0])

    # Run filter over data and collect predictions
    predictions = []
    for z in train_data:
        ukf.predict() # Predicts the next state based on the model.
        ukf.update(z) # Updates the state estimate with the actual closing price
        predictions.append(ukf.x[0])

    r2 = r2_score(train_data, predictions)
```

```
    if r2 > best_r2:
        best_r2 = r2
        best_params = (alpha, beta, kappa, P, Q, R)

print(f"Best R-squared: {best_r2}")
print(f"Best parameters: alpha={best_params[0]}, beta={best_params[1]}, kappa={b
```

```
Best R-squared: 0.997487442028565
Best parameters: alpha=0.001, beta=3.0, kappa=0, P=10.0, Q=1.0, R=0.01


Execution time for grid search: 0 hours, 19 minutes, 14 seconds
```

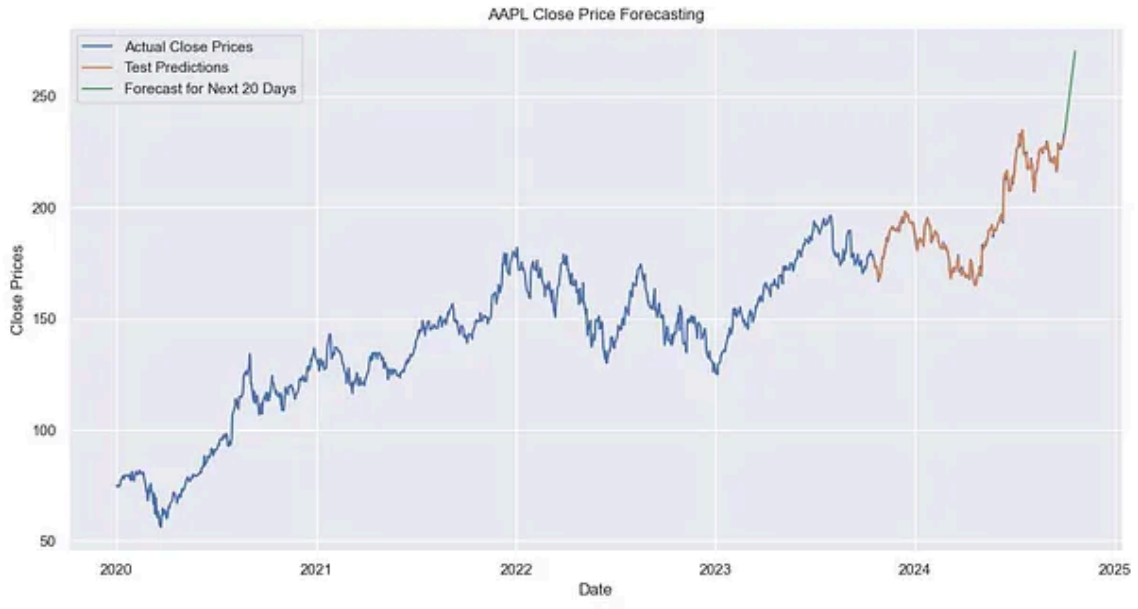
The UKF model prediction is "based on the current estimate and the system's dynamics".

Apply the best parameters to the UKF model for prediction. These parameters may vary depend on type of time-series (here based on AAPL) and its length.

```
from filterpy.kalman import UnscentedKalmanFilter
from filterpy.kalman import MerweScaledSigmaPoints
from filterpy.common import Q_discrete_white_noise

# UKF setup
dt = 1.0   # Assuming daily interval (or adjust as per your data frequency)
n_dim_state = 2  # price and velocity
n_dim_meas = 1  # only measuring price

# Define the state transition function
def fx(x, dt):
    return np.array([x[0] + dt * x[1], x[1]])

# Define the measurement function
def hx(x):
```

```
    return np.array([x[0]])



# Assuming you have found the best parameters from the grid search
#
best_params = {'alpha': 0.001, 'beta': 3.0, 'kappa': 0, 'P': 10, 'Q': 1.0, 'R':

# Initialize the UKF with the best parameters
alpha, beta, kappa = best_params['alpha'], best_params['beta'], best_params['kap
P, Q, R = best_params['P'], best_params['Q'], best_params['R']

points = MerweScaledSigmaPoints(n=n_dim_state, alpha=alpha, beta=beta, kappa=kap
ukf = UnscentedKalmanFilter(dim_x=n_dim_state, dim_z=n_dim_meas, fx=fx, hx=hx, d
ukf.P = np.eye(n_dim_state) * P
#ukf.Q = np.eye(n_dim_state) * Q
ukf.Q = Q_discrete_white_noise(dim=n_dim_state, dt=dt, var=0.004) * Q
ukf.R = np.eye(n_dim_meas) * R
ukf.x = np.array([train_data[0], 0])

# Fit the model to the training data
train_predictions = []
for z in train_data:
    ukf.predict()
    ukf.update(z)
    train_predictions.append(ukf.x[0])


# Evaluate the performance on the test set
test_predictions = []
for z in test_data:
    ukf.predict()
    ukf.update(z)
    test_predictions.append(ukf.x[0])


# Forecast the next n days
filtered_states = []
# Fit the model to the test data
for z in test_data:
    ukf.predict()
    ukf.update(z)
    filtered_states.append(ukf.x.copy())

n_forecast = 15
predictions = []
last_state = filtered_states[-1]
for _ in range(n_forecast):
    last_state = fx(last_state, dt)
    predictions.append(last_state[0])
```

```
# Plot the results
plt.figure(figsize=(14, 7))
plt.plot(stock_data.index, close_prices, label='Actual Close Prices')
plt.plot(stock_data.index[-len(test_data):], test_predictions, label='Test Predi
plt.plot(pd.date_range(start=stock_data.index[-1], periods=n_forecast+1, inclusi
plt.legend()
plt.xlabel('Date')
plt.ylabel('Close Prices')
plt.title(f'{ticker_symbol} Close Price Forecasting')
plt.show()
```




```
# Calculate error metrics
train_mae = mean_absolute_error(train_data, train_predictions)
train_mse = mean_squared_error(train_data, train_predictions)
train_rmse = np.sqrt(train_mse)
train_mape = mean_absolute_percentage_error(train_data, train_predictions)
train_r2 = r2_score(train_data, train_predictions)

test_mae = mean_absolute_error(test_data, test_predictions)
test_mse = mean_squared_error(test_data, test_predictions)
test_rmse = np.sqrt(test_mse)
test_mape = mean_absolute_percentage_error(test_data, test_predictions)
test_r2 = r2_score(test_data, test_predictions)
```

```
metric_data = {
    "Metric": ["MAE", "MSE", "RMSE", "MAPE", "R2"],
    "Train": [train_mae, train_mse, train_rmse, train_mape, train_r2],
    "Test": [test_mae, test_mse, test_rmse, test_mape, test_r2]
}

# Convert to dataframe
metric_df = pd.DataFrame(metric_data)

print(metric_df)
```

```
  Metric     Train      Test
0    MAE  0.643305  0.653400
1    MSE  0.719719  0.794522
2   RMSE  0.848363  0.891359
3   MAPE  0.004859  0.003339
4     R2  0.999312  0.998015
```

Apply the best parameters to the UKF model for prediction without visual (Graph)

I split the data into training and testing sets for grid search and to visualize the model's performance. For actual prediction, however, splitting the data is not necessary.

Use the last portion of closing price as the initial points (current estimate).

```
start_data = close_prices['Close'][-50:]  # 50 is arbitrary
```

```
from filterpy.kalman import UnscentedKalmanFilter
from filterpy.kalman import MerweScaledSigmaPoints
from filterpy.common import Q_discrete_white_noise
```

```
# UKF setup
dt = 1.0   # Assuming daily interval (or adjust as per your data frequency)
n_dim_state = 2  # price and velocity
n_dim_meas = 1  # only measuring price

# Define the state transition function
def fx(x, dt):
    return np.array([x[0] + dt * x[1], x[1]])

# Define the measurement function
def hx(x):
    return np.array([x[0]])
#
# Assuming you have found the best parameters from the grid search
# ================================================================
#
#
# Best parameters: alpha=0.001, beta=3.0, kappa=0, P=10.0, Q=1.0, R=0.01
best_params = {'alpha': 0.001, 'beta': 3.0, 'kappa': 0, 'P': 10, 'Q': 1.0, 'R':

# Initialize the UKF with the best parameters
alpha, beta, kappa = best_params['alpha'], best_params['beta'], best_params['kap
P, Q, R = best_params['P'], best_params['Q'], best_params['R']

points = MerweScaledSigmaPoints(n=n_dim_state, alpha=alpha, beta=beta, kappa=kap
ukf = UnscentedKalmanFilter(dim_x=n_dim_state, dim_z=n_dim_meas, fx=fx, hx=hx, d
ukf.P = np.eye(n_dim_state) * P
#ukf.Q = np.eye(n_dim_state) * Q
ukf.Q = Q_discrete_white_noise(dim=n_dim_state, dt=dt, var=0.004) * Q
ukf.R = np.eye(n_dim_meas) * R
#
# using last portion of close_prices dataframe as current estimate
#
ukf.x = np.array([start_data[0], 0])


# Forecast the next n days
filtered_states = []
# Fit the model to the start_data
for z in start_data:
    ukf.predict()
    ukf.update(z)
    filtered_states.append(ukf.x.copy())

n_forecast = 15  # predict 15 points forward
predictions = []
last_state = filtered_states[-1]    # last value from start_data is on 09/30/202
for _ in range(n_forecast):
    last_state = fx(last_state, dt)
    predictions.append(last_state[0])
```

Compare the predicted values:

```
new_close_prices = stock_data['Close'][-15:]
new_df = pd.DataFrame({'Close Prices': new_close_prices,'Predicted values': pred
new_df
```

```
            Close Prices Predicted values Difference
Date
2024-10-01 226.21     233.510240       -7.300240
2024-10-02 226.78     235.447759       -8.667759
2024-10-03 225.67     237.385279       -11.715279
2024-10-04 226.80     239.322798       -12.522798
2024-10-07 221.69     241.260317       -19.570317
2024-10-08 225.77     243.197837       -17.427837
2024-10-09 229.54     245.135356       -15.595356
2024-10-10 229.04     247.072875       -18.032875
2024-10-11 227.55     249.010395       -21.460395
2024-10-14 231.30     250.947914       -19.647914
2024-10-15 233.85     252.885434       -19.035434
2024-10-16 231.78     254.822953       -23.042953
2024-10-17 232.15     256.760472       -24.610472
2024-10-18 235.00     258.697992       -23.697992
2024-10-21 236.48     260.635511       -24.155511
```

The first prediction value, 233.51, on 2024-10-01, is based on the previous day's closing price (233.00 on 2024-09-30) as the "current estimate" for forecasting. That initial predicted value is then used to generate the next prediction, and the process continues iteratively. As the prediction horizon extends, the deviation naturally grows unless real-time data is available.

If real-time data is available, set n_forecast to 1 so the model predicts only the next point's movement.

As per a reader's request, I placed the test data and test predictions results side-by-side to highlight the high accuracy of the model.

```
test_dates = stock_data.index[-len(test_data):]
test_df = pd.DataFrame(test_data, columns=['Test Data'], index=test_dates)

# Create a new DataFrame to concatenate
test_pred_df = pd.DataFrame({'Test Predicted': test_predictions}, index=test_dat

# Concatenate the new DataFrame with the test data
combined_test_pred_df = pd.concat([test_df, test_pred_df], axis=1)

print(combined_test_pred_df)
```

```
            Test Data  Test Predicted
Date
2023-10-18  175.839996      176.094275
2023-10-19  175.460007      175.319651
2023-10-20  172.880005      173.296142
2023-10-23  173.000000      172.671607
2023-10-24  173.440002      172.921594
...                ...             ...
2024-09-24  227.369995      227.873940
2024-09-25  226.369995      226.978530
2024-09-26  227.520004      227.347049
2024-09-27  227.789993      227.702937
2024-09-30  233.000000      231.572719

[239 rows x 2 columns]
```

Disclaimer:
The information presented herein is provided solely for informational purposes and should not be construed as financial or investment advice. Any investment decisions or actions taken based on this information are made at your own discretion and risk. Stock prices and investment values are subject to fluctuation due to various market factors and conditions.
I strongly recommend consulting with a qualified financial professional for personalized guidance tailored to your individual financial situation and objectives before making any investment decisions.

Further enhancement:
1. This is simple model. fx (state transition function), assuming constant velocity, and hx (Measurement Function) is simply returns the state as the measurement (volatility measurement is ignored). Hence, the predicted values exhibit linearity.
2. UFK still relies on Gaussian noise assumptions. Consider “Particle Filter” for non-Gaussian noise.

Additional articles about Particle Filter:
Tracking a self-driving car with Particle Filter
Particle Filter on Localisation
Object Tracking Using Particle Filter,
Particle Filter Part 1, Part 2, Part 3, Part 4
Particle filter

Thank you for reading.

Recommended books:
Kalman Filter from the Ground Up, Alex Becker (2023, kalmanfilter.net)
Unscented Kalman Filter Made Easy, William Franklin, (2024 Independently published)
Kalman-and-Bayesian-Filters-in-Python, by Roger Labbe, (2022 GitHub:rlabbe)

Reference:
[1] Tracking pedestrians for self driving cars
[2] Using the Unscented Kalman Filter on Stock Prices
[3] Kalman Filter: Historical Overview and Review of Its Use in Robotics 60 Years after Its Creation
[4] Unscented Kalman Filter for Noisy Multivariate Financial Time-Series Data

Written by Simon Leung
229 followers · 4 following

Edit profile

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.